Krishi Mitra — Full Production RAG Pipeline Audit

This document audits every stage of the RAG pipeline against industry best practices. Rated on a 4-point scale:

Rating	Meaning
✅ **Gold**	Industry standard / production-ready
🟢 **Good**	Solid implementation, minor gaps
🟡 **Adequate**	Works for thesis scope, not production
🔴 **Missing**	Not implemented, would block production

---

1. DATA INGESTION & DOCUMENT LOADING

Practice	Status	Notes
Source format handling	🟡	Single JSON file only. No PDF, Word, web scraping, or multi-format support.
Document parsing	🟡	`flatten_value()` recursively renders nested dicts. No OCR, no table extraction, no image handling.
Metadata extraction	🟢	Category inferred from section keys (`pest_management` → "pest"). Good for a structured corpus.
Data provenance tracking	🔴	No source URL, version, or last-updated timestamp stored with chunks.
Incremental updates	🔴	Full rebuild only (`build_database()` deletes and recreates). No delta ingestion.
Deduplication	🔴	No check for duplicate content across updates.

Verdict: Adequate for a single-crop thesis. For production: needs multi-format parsers, incremental ingestion, and provenance tracking.

---

2. TEXT PREPROCESSING & CLEANING

Practice	Status	Notes
Unicode normalization	🟢	`indic_preprocess.py` applies NFC normalization, ZWJ/ZWNJ cleanup. Addresses Kannada agglutination artifacts.
Language detection	🔴	No detection —assumes all text is Kannada/English mixed.
Stopword removal	🔴	Not done. Kannada stopwords (, , , etc.) remain in text.
Stemming / lemmatization	🔴	Not implemented. The architect review flagged this as the core agglutination gap. BGE-M3' s SentencePiece tokenizer provides partial subword coverage, but explicit stemming would help.
Punctuation normalization	🟢	Handled by `indic_preprocess.py` (danda spacing, multiple spaces).
HTML/Markdown stripping	⏳	N/A —source is clean JSON.
PII detection / redaction	🔴	No input sanitization. If a farmer pastes Aadhaar numbers, they go straight into the embedding.

Verdict: Good Unicode normalization, but missing language-aware preprocessing (stopwords, stemming). PII handling is a genuine production gap.

---

3. CHUNKING STRATEGY

Practice	Status	Notes
Fixed-size sliding window	🟡	Original implementation. Simple, fast, but splits mid-sentence and ignores structure.
Field-aware chunking	🟢	One chunk per logical JSON unit (pest, disease, schedule). Respects document structure.

Table 4 – continued

Practice	Status	Notes
Semantic chunking	🟢	**NEW** —embedding-based boundary detection. Algorithmic, structure-agnostic, generalizes to unstructured text. This is the gold-standard approach for modern RAG.
Sentence-boundary awareness	🟢	Semantic chunker splits at Kannada danda ( ) and periods when hard ceiling is hit.
Chunk size tuning	🟡	`max_chunk_chars=1200` is reasonable but not empirically tuned. No ablation on chunk size vs retrieval quality.
Chunk overlap	🟡	200-char overlap in legacy mode. Semantic chunker doesn' t use overlap —each boundary is a clean split.
Parent-document retrieval	🔴	Not implemented. If a chunk is small, the full parent document isn' t retrieved for additional context.
Hierarchical chunking	🔴	Not implemented. No "summary chunk → detail chunk" hierarchy.

Verdict: Semantic chunking brings this to **Good** tier. For production: add parent-document retrieval and chunk-size ablation.

---

4. EMBEDDING MODEL

Practice	Status	Notes
Model choice	🟢	**BGE-M3** is state-of-the-art for multilingual retrieval. Dense + sparse + colbert in one model. Excellent choice.

Table 5 – continued

Practice	Status	Notes
Multilingual support	🟢	BGE-M3 supports 100+ languages including Kannada. XLM-R-based SentencePiece tokenizer handles subwords.
Fine-tuning on domain	🔴	Not done. Using off-the-shelf weights. For a specialized agri corpus, domain-adapted embeddings would improve recall.
Embedding dimension	🟢	1024-d dense + sparse lexical weights. Standard for BGE-M3.
Batch encoding	🟢	`embed_model.encode(chunks, ...)` batches all chunks at once. Efficient.
Query vs document encoding	🟡	Same model for both. BGE-M3 is symmetric (designed for this), but some systems use asymmetric models (e.g., E5-mistral for queries, BGE for docs).
Embedding caching	🔴	No cache. Same query re-embedded every time. For production, cache query embeddings in Redis/ Memcached.
Embedding versioning	🔴	No version tracking. If you switch from BGE-M3 to another model, old embeddings are incompatible but not marked.

Verdict: Model choice is excellent. Production gaps: no fine-tuning, no caching, no versioning.

---

5. VECTOR STORE & INDEXING

Practice	Status	Notes
Vector database	🟡	**Qdrant** in local file mode. Good database, but file-mode is single-process and requires exclusive access. For production: use Qdrant server mode or cloud.
Hybrid search	🟢	**Dense + sparse + RRF fusion** —gold standard. BGE-M3 provides both vectors; Qdrant handles fusion.
Metadata filtering	🟢	**NEW** —category payload filtering with `should` boost. Uses information already computed.
Payload schema	🟡	Only `category` and `text` stored. No timestamp, no source, no chunk hierarchy.
Index optimization	🔴	No HNSW parameter tuning (ef, m). Default Qdrant settings used.
Replication / HA	🔴	File-mode Qdrant has no replication. Single point of failure.
Backup strategy	🔴	No automated backup of the Qdrant directory.
Collection versioning	🔴	Single collection. No A/B testing of different chunking strategies side-by-side.

Verdict: Hybrid search is excellent. Infrastructure is thesis-appropriate but not production-ready.

---

6. QUERY PREPROCESSING

Practice	Status	Notes
Query normalization	🟢	**NEW** —`embed_query()` applies Indic NLP normalization before embedding. Matches corpus preprocessing.

Table 7 – continued

Practice	Status	Notes
Query expansion	🔴	Not implemented. No synonym expansion, no hyponym/hypernym expansion for agri terms.
Query rewriting / condensation	🔴	**TODO #14** –chat history is passed to generation but NOT to retrieval. Follow-up queries like “ ?” retrieve nothing because the embedding sees only the fragment.
Spelling correction	🔴	No autocorrect for Kannada. A typo in a chemical name (→) would miss entirely.
Stopword removal from query	🔴	Not done. “ ?” — , , etc. are stopwords that dilute the embedding.
Query classification	🟢	Router classifies into price/ disease/pest/fertilizer/ general. Good for routing and category boosting.
Intent detection	🟡	Router does basic classification, but no fine-grained intent (e.g., “prevent” vs “treat” vs “identify”).

Verdict: Query normalization is good. Missing: expansion, condensation, spelling correction, stopword removal. The condensation gap is the most consequential for user experience.

---

7. RETRIEVAL STRATEGY

Practice	Status	Notes
Dense retrieval	🟢	Cosine similarity on 1024-d BGE-M3 dense vectors. Standard.
Sparse retrieval (learned)	🟢	BGE-M3 lexical weights — learned sparse representation. Modern alternative to BM25.

Table 8 – continued

Practice	Status	Notes
Sparse retrieval (statistical)	🟢	**NEW** –BM25 via `rank_bm25`. Pure statistical, corpus-calibrated, formula-explainable.
Hybrid fusion	🟢	RRF (Reciprocal Rank Fusion) of dense + sparse. Industry standard.
BM25 + dense fusion	🟢	**NEW** –RRF of BM25 + dense. Strong for exact-term queries.
Multi-vector retrieval	🔴	Not implemented. Each chunk has one dense vector. No multi-vector per document.
Max marginal relevance	🔴	Not implemented. Retrieval optimizes for similarity, not diversity. Could return 5 chunks about the same pest.
Recency filtering	⏳	N/A –static corpus, no time dimension.
Geographic filtering	🔴	Not implemented. A query about "red soil in North Karnataka" would retrieve the same chunks as "red soil in South Karnataka" even if the advice differs.

Verdict: Excellent for a thesis. Dense + learned sparse + statistical sparse + hybrid fusion covers all major retrieval paradigms. Production gap: no diversity control (MMR).

---

8. RERANKING

Practice	Status	Notes
Cross-encoder reranker	🟡	**BGE-Reranker-v2-m3** loaded but **disabled by default** (`ENABLE_RERANKER=false`). The P1.3 decision found it added ~30s latency on CPU with minimal MRR gain on a tiny corpus.

Table 9 – continued

Practice	Status	Notes
Reranker threshold	🟡	`RERANK_THRESHOLD=0.5` — filters out low-confidence reranked results. Not empirically tuned.
Latency-aware gating	🟢	Reranker is conditionally enabled via env var. Good engineering practice.
ColBERT-style late interaction	🔴	BGE-M3 supports ColBERT vectors, but not used in retrieval. Late interaction would improve precision without full reranker latency.
Learned ranking (LTR)	🔴	Not implemented. No gradient-boosted or neural LTR model trained on click/feedback data.

Verdict: Reranker exists but is off. For production: enable it on GPU, or implement ColBERT late interaction for a latency/precision trade-off.

---

9. CONTEXT ASSEMBLY & PROMPT ENGINEERING

Practice	Status	Notes
Context window size	🟡	Top-5 chunks. Not tuned — could be 3 or 7 depending on chunk size and question complexity.
Context ordering	🟡	Retrieved order (RRF rank). No re-ordering by relevance or recency.
Context deduplication	🔴	Not implemented. If two chunks contain overlapping text, both are passed to the LLM, wasting tokens.
Context truncation	🔴	No explicit truncation. If 5 chunks $\times$ 1200 chars = 6000 chars, that' s ~1500 tokens. Could exceed context window for smaller models.

Table 10 – continued

Practice	Status	Notes
System prompt	🟢	Clear, strict instructions: "Do not repeat the question. Answer using only the provided context."
Prompt injection defense	🔴	No input sanitization. A malicious user could inject instructions via the query.
Few-shot examples	🔴	Not used. The prompt is zero-shot. Few-shot examples in Kannada could improve output quality.
Chain-of-thought prompting	🔴	Not used. For complex multi-step questions ("if I have red soil and want to prevent both pests and diseases, what should I do?"), CoT would improve reasoning.
Structured output (JSON mode)	🔴	**TODO #40** —Not implemented. Dosages are free-text. For safety, structured extraction (chemical, dose, unit, timing) would be stronger.
Temperature control	🟢	`temperature=0.0` — deterministic, reduces eval variance. Good for factual QA.
Max tokens limit	🟢	`INTERACTIVE_MAX_PREDICT=250` / `EVAL_MAX_PREDICT=500`. Appropriate bounds.

Verdict: Prompt is solid but basic. Missing: context truncation, deduplication, few-shot, CoT, structured output, prompt injection defense.

---

10. GENERATION / LLM

Practice	Status	Notes
Model abstraction	🟢	**NEW** —`llm_client.py` supports Groq, OpenRouter, Ollama. Clean backend switching.
Model selection	🟡	`openai/gpt-oss-20b` (suggested default). Need to verify this is active on Groq.
Self-evaluation bias avoidance	🟢	**NEW** —OpenRouter backend enables cross-judge (different provider judges Groq output).
Streaming	🔴	**TODO #15** —Full-response blocking calls only. Streaming would improve perceived latency.
Retry logic	🔴	No automatic retry on transient failures (rate limits, timeouts).
Fallback model	🔴	If Groq fails, no automatic fallback to OpenRouter or Ollama.
Request timeout	🟡	`INTERACTIVE_TIMEOUT=120` seconds. Hard timeout with fallback message. Good.
Token usage tracking	🔴	No logging of prompt tokens, completion tokens, or cost per query.
Response caching	🔴	No cache. Identical queries re-call the LLM every time.

Verdict: Abstraction is excellent. Production gaps: no streaming, no retry/fallback, no token tracking, no caching.

---

11. POST-GENERATION VALIDATION

Practice	Status	Notes
Semantic faithfulness judge	🟢	LLM-as-judge scores answer against context (0.0–1.0). Good approach.
Cross-judge	🟢	**NEW** —OpenRouter enables different-model judging. Reduces self-preference bias.

Table 12 – continued

Practice	Status	Notes
Numeric faithfulness checker	🟢	**NEW** —Regex extracts number+unit from answer, cross-references against context. Closes highest-consequence safety gap.
Hallucination detection (claim-level)	🔴	Not implemented. No NLI (Natural Language Inference) model checking each claim independently.
Citation verification	🔴	Not implemented. The answer doesn' t cite which chunk each fact came from.
Answer consistency check	🔴	Not implemented. No check that the answer is internally consistent (e.g., "apply 10g" and "apply 100g" in the same answer).
Output format validation	🔴	Not implemented. No JSON schema validation for structured outputs.

Verdict: Faithfulness + numeric checker is strong for a thesis. Production would need claim-level verification and citations.

---

12. SAFETY & GUARDRAILS

Practice	Status	Notes
Hard refusal for low confidence	🟢	`HARD_REFUSAL_THRESHOLD` blocks answers when retrieval score is too low.
Safety-critical category gating	🟢	Pest/disease/fertilizer queries require higher confidence (`CONFIDENT_SEARCH_THRESHOLD`).
Faithfulness gate	🟢	Answers with semantic score < 0.5 are replaced with refusal.
Numeric faithfulness gate	🟢	**NEW** —Answers with unsupported numbers are blocked.

Table 13 – continued

Practice	Status	Notes
Combined gate logic	🟢	**NEW** —Semantic OR numeric failure triggers refusal. Robust.
Threshold empirical justification	🟢	**NEW** —`threshold_sweep.py` tests candidates against gold labels. Reports TPR/TNR/F1.
Adversarial testing	🟡	`refusal_test.py` tests unanswerable questions. Good start.
Prompt injection defense	🔴	No sanitization. Adversarial text in user input goes straight to LLM.
Toxicity / harmful content filter	🔴	No output filtering beyond the faithfulness gate.
Rate limiting	🔴	No API rate limits. A malicious user could spam the endpoint.
Input length limits	🟡	FastAPI validates non-empty query, but no max length check. A 10,000-char query would be processed.
Escalation message	🟢	Appends Kisan Call Centre number (1800-180-1551) for medium-confidence answers. Good for farmer-facing use case.

Verdict: Strong safety layer for a thesis. The numeric checker + combined gate + threshold sweep is genuinely impressive. Production gaps: prompt injection, rate limiting, input length limits.

---

13. MULTI-TURN / CONVERSATIONAL RAG

Practice	Status	Notes
Chat history storage	🟢	MongoDB stores messages with timestamps. Good.
History in generation	🟢	Last 3 messages passed to LLM as context. Standard.

Table 14 – continued

Practice	Status	Notes
History in retrieval	🔴	**TODO #14** —Chat history is NOT used for query rewriting/condensation. Follow-up queries retrieve nothing.
Session management	🟡	`session_id` passed by client. No auth, no session expiry.
Context window management	🔴	History grows unbounded in MongoDB. No pruning of old messages.
Conversation memory (summarization)	🔴	No long-term memory. After 3 messages, older context is lost.

Verdict: Basic chat works. The missing query condensation is the single biggest UX gap.

---

14. EVALUATION METRICS

Practice	Status	Notes
IR metrics (recall@k, MRR, nDCG)	🟢	**Gold standard** for retrieval evaluation. Properly implemented against gold labels.
Bucketed analysis	🟢	**NEW** —Exact-term vs semantic query split. Shows where BM25 wins and where dense wins.
Statistical significance	🟡	Reports mean, std, 95% CI. No formal hypothesis testing (t-test, bootstrap).
Generation metric (chrF)	🟢	Script-agnostic, better than ROUGE-L for Kannada. Good choice.
Embedding similarity	🟡	Used as secondary metric. chrF is primary —correct prioritization.
Faithfulness score (LLM judge)	🟢	0.0–1.0 scale, cross-judge capable. Good.
Metric sanity check	🟢	`diagnose_metrics.py` computes self-score vs unrelated-score. Proves metrics have usable range.

Table 15 – continued

Practice	Status	Notes
Controlled variable design	🟢	`build_contexts.py` freezes retrieval so generation differences are isolated. Excellent experimental design.
n=16 sample size	🟡	Too small for statistical confidence. **TODO #18** – expand to 40–100.
Human evaluation	🔴	Not done. No human annotators rating answer quality.
A/B test framework	🔴	No framework for comparing two configs with real users.
Regression test	🟡	`refusal_test.py` exists but not wired to CI. **TODO #12**.
Latency benchmarking	🟢	All eval scripts report latency. Good.
Cost tracking	🔴	No tracking of API costs per evaluation run.

Verdict: Excellent evaluation rigor for a thesis. IR metrics + controlled variables + metric sanity checks are genuinely impressive. Sample size is the main weakness.

---

15. OBSERVABILITY & LOGGING

Practice	Status	Notes
Structured logging	🔴	`print()` statements only. No JSON-structured logs, no log levels (DEBUG/INFO/ERROR).
Request tracing	🔴	No trace ID per request. Can't follow a single query through router → retrieval → generation → judge.
Latency breakdown	🟡	Some latency logging (Groq call time), but no end-to-end breakdown (router + retrieval + generation + judge).
Error tracking	🔴	No Sentry, no centralized error collection.

Table 16 – continued

Practice	Status	Notes
Metrics dashboard	🔴	No Grafana/Datadog. No tracking of QPS, P95 latency, error rate.
Query analytics	🔴	No logging of what users actually ask. Can' t discover knowledge gaps.
Feedback loop	🔴	No thumbs up/down on answers. No mechanism to improve from user feedback.

Verdict: Print-based debugging is fine for a thesis. Production needs structured logging, tracing, and a feedback loop.

---

16. DEPLOYMENT & INFRASTRUCTURE

Practice	Status	Notes
Containerization	🟡	**TODO #9** –Docker mentioned in plans but not implemented yet.
Health check endpoint	🟢	`/health` returns `{"status": "ok"}`. Good.
CORS configuration	🟢	Configured for `localhost:3000`. Appropriate for dev.
API documentation	🟡	FastAPI auto-generates OpenAPI docs at `/docs`. Good.
Authentication	🔴	No auth on any endpoint. Anyone can hit `/chat`.
Rate limiting	🔴	No rate limits.
Input validation	🟡	Pydantic models validate `query` and `session_id`. No max length, no content filtering.
Graceful shutdown	🟢	`close_db()` on shutdown. Good.
Environment configuration	🟢	`.env` with all vars documented. Good.
Secrets management	🟡	`.env` file. For production: use a secrets manager (AWS Secrets Manager, Doppler, etc.).

Table 17 – continued

Practice	Status	Notes
CI/CD	🔴	**TODO #12** –GitHub Action mentioned but not implemented.
Load balancing	🔴	Single FastAPI process. No horizontal scaling.
Database connection pooling	🟡	Motor (async MongoDB client) handles pooling. Qdrant file-mode is single-process.

Verdict: Dev setup is solid. Production needs auth, rate limiting, Docker, CI/CD, and scaling.

Summary Scorecard

Category	Score	Key Strengths	Key Gaps
Chunking	🟢 Good	Semantic chunking + Indic NLP normalization	No parent-document retrieval, no chunk-size ablation
Embedding	🟢 Good	BGE-M3 (SOTA multilingual)	No domain fine-tuning, no caching
Retrieval	🟢 Good	Dense + sparse + BM25 + hybrid fusion	No MMR diversity, no query expansion
Reranking	🟡 Adequate	Cross-encoder exists, conditionally enabled	Disabled by default, no ColBERT late interaction
Generation	🟡 Adequate	Multi-backend abstraction, cross-judge	No streaming, no retry, no caching
Safety	🟢 Good	Hard refusal + faithfulness + numeric gate + threshold sweep	No prompt injection defense, no rate limiting
Evaluation	🟢 Good	IR metrics + controlled variables + metric sanity + bucketed analysis	n=16 too small, no human eval, no CI
Multi-turn	🔴 Missing	Basic chat history	No query condensation –biggest UX gap
Observability	🔴 Missing	Print debugging	No structured logs, no tracing, no feedback loop
Deployment	🟡 Adequate	Health check, CORS, .env	No auth, no Docker, no CI/CD

Overall Verdict

For a thesis defense: This is a **strong, above-average project**. The semantic chunking, numeric faithfulness checker, hybrid retrieval with BM25 ablation, and controlled-variable evaluation design are genuinely impressive. Most CS thesis projects don't have this level of evaluation rigor or safety awareness.

For production deployment: This is a **solid prototype** that needs 3–6 months of engineering work to reach production. The core RAG logic is sound, but infrastructure (auth, scaling, observability) and UX (streaming, query condensation, feedback) are missing.

What to Highlight in Your Defense

1. **"We use semantic boundary detection for chunking"** — the embedding model itself finds topic shifts, not hardcoded rules.
2. **"We have three retrieval paradigms"** — dense (neural), learned sparse (BGE-M3 lexical), and statistical sparse (BM25), with evidence for when each wins.
3. **"Our safety layer has two independent checks"** — semantic faithfulness + numeric cross-reference. Either one can block a hallucinated answer.
4. **"Our thresholds are empirically justified"** — not magic numbers, but F1-optimized against gold-labeled data.
5. **"We use controlled-variable experimental design"** — retrieval is frozen when comparing generation models, so differences are attributable to generation, not retrieval variance.

What to Admit as Future Work

1. **Query condensation for multi-turn** — "This is our biggest UX gap. We pass history to generation but not to retrieval."
2. **Streaming responses** — "Blocking calls are fine for evaluation but poor for user experience."
3. **Production infrastructure** — "Auth, rate limiting, Docker, and CI/CD are deliberately deferred for this prototype scope."
4. **Larger eval set** — "n=16 is sufficient for directional comparison but not for statistical significance. We plan to expand to 100+ questions from the Kisan Call Centre dataset."